



Laboratory Assessment IV
PEC-2 Laboratory: Full Stack Development - BIT26PE05
To design and develop a RESTful API
Varad Sushant Deshmukh [123B1F022] TY-IT-A, PCCOE College

AIM:

To design and develop a RESTful API using Spring Boot that performs CRUD operations for managing employee records, along with unit testing using JUnit.

OBJECTIVES

- Build REST endpoints for employee management
- Implement Create, Read, Update, Delete operations
- Use Spring Boot for backend development
- Ensure proper request/response handling (JSON)
- Write unit tests using JUnit to validate functionality

API Design:

- **Endpoints:**

Method	Endpoint	Description
GET	/employees	Get all employees
GET	/employees/{id}	Get employee by ID
POST	/employees	Add new employee
PUT	/employees/{id}	Update employee
DELETE	/employees/{id}	Delete employee

Controller

```
package com.example.crudTesting.controller;
import com.example.crudTesting.Service.EmployeeService;
import com.example.crudTesting.model.Employee;
import org.springframework.web.bind.annotation.*;

import java.util.List;
@RestController
@RequestMapping("/employees")
public class EmployeeController {
    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping
    public List<Employee> getAllEmployees() {
        return service.getAllEmployees();
    }

    @GetMapping("/{id}")
    public Employee getEmployee(@PathVariable int id) {
        return service.getEmployeeById(id);
    }

    @PostMapping
    public Employee addEmployee(@RequestBody Employee employee) {
        return service.saveEmployee(employee);
    }

    @PutMapping("/{id}")
    public Employee updateEmployee(@PathVariable int id, @RequestBody Employee
employee) {
        employee.setId(id);
        return service.saveEmployee(employee);
    }

    @DeleteMapping("/{id}")
    public void deleteEmployee(@PathVariable int id) {
        service.deleteEmployee(id);
    }
}
```

Entity

```
package com.example.crudTesting.model;
import jakarta.persistence.*;
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private String role;
    private double salary;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getRole() {
        return role;
    }
    public void setRole(String role) {
        this.role = role;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

Repository

```
package com.example.crudTesting.repository;

import com.example.crudTesting.model.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}

Service

package com.example.crudTesting.Service;

import com.example.crudTesting.model.Employee;
import com.example.crudTesting.repository.EmployeeRepository;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    public Employee getEmployeeById(int id) {
        return repository.findById(id).orElse(null);
    }

    public Employee saveEmployee(Employee employee) {
        return repository.save(employee);
    }

    public void deleteEmployee(int id) {
        repository.deleteById(id);
    }
}
```

JUnit Testing:

```
package com.example.crudTesting;

import com.example.crudTesting.Service.EmployeeService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

@WebMvcTest
public class EmployeeControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private EmployeeService service;

    @Test
    void testGetAllEmployees() throws Exception {
        mockMvc.perform(get("/employees"))
            .andExpect(status().isOk());
    }
}
```

}

pom.xml

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

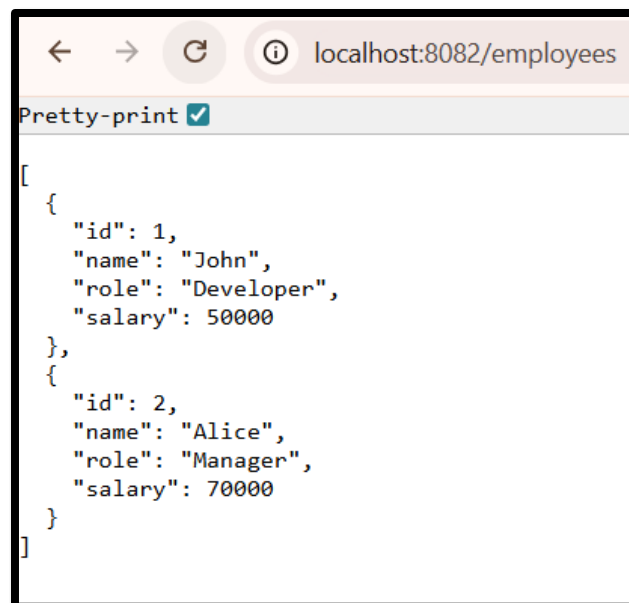
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

```
</dependencies>
```

@GETMAPPING:



```
[
  {
    "id": 1,
    "name": "John",
    "role": "Developer",
    "salary": 50000
  },
  {
    "id": 2,
    "name": "Alice",
    "role": "Manager",
    "salary": 70000
  }
]
```

@POSTMAPPING:

```
INSERT INTO EMPLOYEES (NAME, ROLE, SALARY) VALUES ('John', 'Developer', 50000);
```

```
INSERT INTO EMPLOYEES (NAME, ROLE, SALARY) VALUES ('Alice', 'Manager', 70000);
```

```
select * from employees;
```

```
select * from employees;
```

ID	NAME	ROLE	SALARY
1	John	Developer	50000.0
2	Alice	Manager	70000.0

(2 rows, 6 ms)

Edit

@PUTMAPPING:

```
UPDATE EMPLOYEES SET SALARY=100000 WHERE NAME='John';
select *from employees;
```

```
UPDATE EMPLOYEES SET SALARY=100000 WHERE NAME='John';
Update count: 1
(1 ms)
```

```
select *from employees;
```

ID	NAME	ROLE	SALARY
1	John	Developer	100000.0
2	Alice	Manager	70000.0

(2 rows, 3 ms)

@DELETEMAPPING:

```
DELETE FROM EMPLOYEES WHERE NAME='Alice';
select *from employees;
```

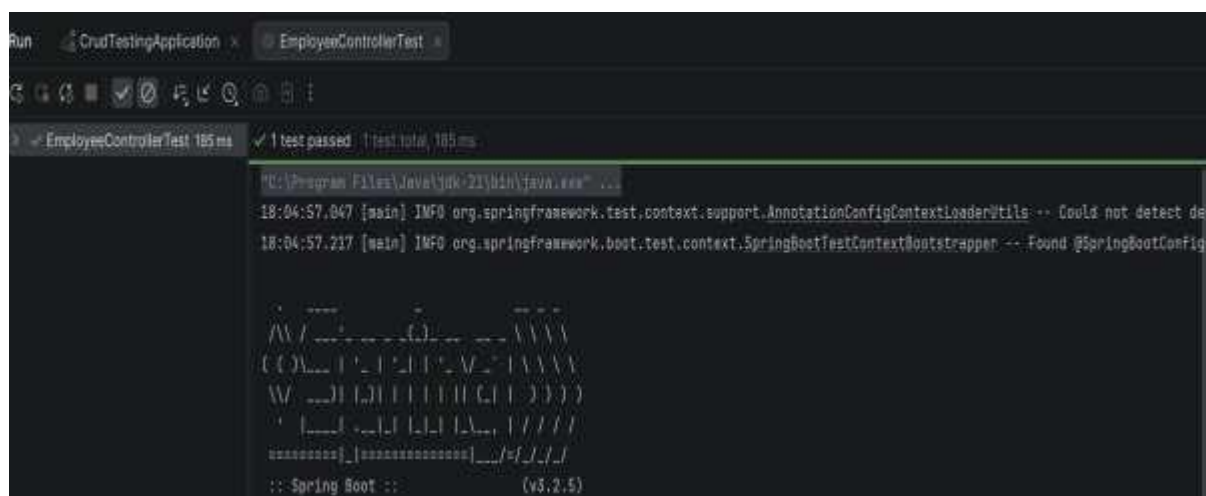
```
DELETE FROM EMPLOYEES WHERE NAME='Alice';
Update count: 0
(0 ms)
```

```
select *from employees;
```

ID	NAME	ROLE	SALARY
1	John	Developer	100000.0

(1 row, 1 ms)

TEST CASE RESULT:



Conclusion:

The RESTful API successfully demonstrates CRUD operations for managing employee records using Spring Boot. The layered architecture ensures clean separation of concerns, while JUnit testing validates the functionality and reliability of the application. This implementation highlights the importance of structured API design, proper data handling, and testing in building scalable backend systems.